

논문 및 깃허브 링크

Text2NKG: Fine-Grained N-ary Relation Extraction for N-ary relational Knowledge Graph Construction

<https://neurips.cc/virtual/2024/poster/94929>

<https://github.com/LHRLAB/Text2NKG>

1. 서론

RAG의 일종인 GraphRAG는 지식그래프를 사용하는데, 전통적인 지식그래프는 주로 (subject, relation, object) 형태의 binary triple, 즉 노드(subject)와 노드(object)를 연결하는 선분 형태의 엣지(relation)로 사실을 표현한다. 하지만 실제 세계의 지식은 시간, 장소, 원인, 역할 등 여러 부가 정보를 포함하는 경우가 많다. 이러한 복잡한 사실을 표현하기 위해 하나의 관계에 여러 argument(qualifier)를 붙여 나타내는, 즉 엣지가 단일 노드와 노드를 연결하는 선분이 아니라 여러 노드를 포함하는 집합의 형태로 나타나는 하이퍼그래프에 대해 조사했다. NeurIPS 2024에 제출된 Text2NKG가 BERT 기반 인코더와 다양한 헤드를 이용해, 문장으로부터 N-ary hyper-fact를 추출하는 모델 및 학습 프레임워크를 깃허브에 공개하고 있어 구현목표로 했다.

가능한 한 깃허브에 공개된 코드를 그대로 사용하려고 했으나, 실행 환경과 라이브러리 구성이 맞지 않아 apex 기반 FP16 학습을 제거하고 FP32 학습으로 전환했다.

2. Text2NKG 개요

고전적인 지식그래프 구축 과정에는 NER 모델과 RE 모델이 사용된다.

NER(Named Entity Recognition, 개체명 인식): 텍스트 내에서 이름이 있는 개체(사람 이름, 조직, 위치, 시간, 날짜 등)를 식별하고 미리 정의된 범주로 분류하는 작업

RE(Relation Extraction, 관계 추출): 텍스트에서 식별된 개체들 간의 의미론적 관계를 감지하고 분류하는 작업

일반적인 RE 모델은 주로 (subject, relation, object) 형태의 binary triple을 출력하지만, Text2NKG는 시간, 장소, 원인, 역할 등 여러 부가 정보를 포함하는 N-ary fact (NKG hyper-fact)를 출력하도록

RE 모델을 훈련시켰다.

3. 학습 설정 및 실행 방법

3.1 HyperRED 데이터셋

HyperRED는 N-ary relation extraction을 위한 데이터셋으로, 각 라인은 문장과 함께 엔티티, relation 타입, qualifier 정보를 포함한다.

```
{ "sentences": [ ["Apollo", "is", "the", "son", "of", "Zeus", "and", "Leto",  
",", "and", "has", "a", "twin", "sister", ",", "the", "chaste", "huntress",  
"Artemis", "."], "ner": [[[5, 5, "Entity"], [18, 18, "Entity"], [7, 7,  
"Entity"]]], "relations": [[[5, 5, 18, 18, "[r]child", [[7, 7,  
"[k]mother"]]]], "clusters": [], "doc_key": ""}
```

n-ary fact를 여러 개의 3-ary 조각으로 쪼개, 문장에서 (subject, object, qualifier)의 3원소 튜플을 qualifier 개수만큼 추출한 뒤, 세 원소의 순서를 바꿔 $3! = 6$ 개 순열을 만들어 데이터를 증강하고 모델이 다양한 엔티티 순서를 학습하도록 한다.

3.2 학습 방법

```
class BertForACEBothOneDropoutSub(BertPreTrainedModel):  
    def __init__(self, config):  
        super().__init__(config)  
        self.max_seq_length = config.max_seq_length  
        self.num_labels = config.num_labels  
        self.num_ner_labels = config.num_ner_labels  
        self.num_q_labels = config.num_q_labels  
        self.nary_schema = config.nary_schema  
  
        self.bert = BertModel(config)  
        self.dropout = nn.Dropout(config.hidden_dropout_prob)  
  
        self.ner_classifier = nn.Linear(config.hidden_size*2, self.num_ner_labels)  
        if self.nary_schema != "role":  
            self.re_classifier_m1 = nn.Linear(config.hidden_size*2, self.num_labels)  
            self.re_classifier_m2 = nn.Linear(config.hidden_size*2, self.num_labels)  
            self.re_classifier_m3 = nn.Linear(config.hidden_size*2, self.num_labels)  
  
        if self.nary_schema != "hypergraph":  
            self.q_re_classifier_m1 = nn.Linear(config.hidden_size*2, self.num_q_labels)  
            self.q_re_classifier_m2 = nn.Linear(config.hidden_size*2, self.num_q_labels)  
            self.q_re_classifier_m3 = nn.Linear(config.hidden_size*2, self.num_q_labels)  
  
        self.alpha = torch.tensor([config.alpha] + [1.0] * (self.num_labels-1), dtype=torch.float32)  
        self.ner_alpha = torch.tensor([config.alpha] + [1.0] * (self.num_ner_labels-1), dtype=torch.float32)  
        self.q_alpha = torch.tensor([config.q_alpha] + [1.0] * (self.num_q_labels-1), dtype=torch.float32)
```

Relation, qualifier 각각에 대해 3개의 FFN으로 이루어진 classifier인 re_classifier와 q_re_classifier를

사용한다. 대부분의 (subject, object, qualifier)는 relation이 없으므로 relation이 없을 때의 가중치를 낮춘다.

```
class BertForACEBothOneDropoutSub(BertPreTrainedModel):
    def forward(
        outputs = self.bert(
            input_ids,
            attention_mask=attention_mask,
            token_type_ids=token_type_ids,
            position_ids=position_ids,
            head_mask=head_mask,
            inputs_embeds=inputs_embeds,
        )
        hidden_states = outputs[0]
        hidden_states = self.dropout(hidden_states)
        seq_len = self.max_seq_length
        bsz, tot_seq_len = input_ids.shape
        ent_len = (tot_seq_len - seq_len) // 2

        e1_hidden_states = hidden_states[:, seq_len:seq_len+ent_len]
        e2_hidden_states = hidden_states[:, seq_len+ent_len:]

        feature_vector = torch.cat([e1_hidden_states, e2_hidden_states], dim=2)

        ner_prediction_scores = self.ner_classifier(feature_vector)

        # relation_feature
        r_feature_vector = torch.stack([feature_vector]*ent_len, dim=2)
        # r_ner_prediction_scores = self.ner_classifier(r_feature_vector)

        # qualifier_feature
        q_feature_vector = torch.stack([feature_vector]*ent_len, dim=1)
        q_ner_prediction_scores = self.ner_classifier(q_feature_vector)
```

BertForACEBothOneDropoutSub.forward()는 입력 토큰 시퀀스를 BERT 인코더에 통과시켜 토큰별 hidden state를 얻는다(hidden_states). 이후 최대 시퀀스 길이(max_seq_length)와 전체 시퀀스 길이(tot_seq_len)를 이용해, 앞 부분은 실제 단어 토큰 구간, 뒤쪽은 엔티티 슬롯 구간으로 나눈다.

e1_hidden_states와 e2_hidden_states는 각각 두 개의 엔티티 슬롯(hidden state)을 의미하며, 이 둘을 concat한 feature_vector는 각 엔티티에 대한 최종 표현 벡터이다. ner_classifier는 이 feature_vector를 입력으로 받아 엔티티 타입을 분류하는 NER logits(ner_prediction_scores)을 계산한다. 이어서 feature_vector를 엔티티 개수(ent_len)만큼 반복하여 relation용(r_feature_vector),

qualifier용(q_feature_vector) feature 텐서를 만들고, 이후 relation/qualifier 분류기에 사용한다.

```
m1_start_states = hidden_states[torch.arange(bsz), sub_positions[:, 0]]
m1_end_states = hidden_states[torch.arange(bsz), sub_positions[:, 1]]
m1_states = torch.cat([m1_start_states, m1_end_states], dim=-1)

if self.nary_schema != "hypergraph":
    ## qulifier_re
    q_m1_scores = self.q_re_classifier_m1(m1_states) # bsz, num_label
    q_m2_scores = self.q_re_classifier_m2(r_feature_vector) # bsz, ent_len, num_label
    q_m3_scores = self.q_re_classifier_m3(q_feature_vector)
    q_re_prediction_scores = q_m1_scores.unsqueeze(1).unsqueeze(2) + q_m2_scores + q_m3_scores
```

sub_positions에는 각 배치 샘플마다 주어(subject) 엔티티의 시작/끝 토큰 인덱스가 저장되어 있다. 모델은 이 위치에 해당하는 hidden state를 추출한 후(m1_start_states, m1_end_states), 두 벡터를 이어 붙여 subject 표현(m1_states)을 만든다.

관계 분류기는 세 개의 선형 계층 re_classifier_m1/2/3로 구현되어 있다. m1_states는 주어에 대한 전역 정보, r_feature_vector는 (sub, obj) 조합에 대한 정보, q_feature_vector는 (sub, q) 조합에 대한 정보를 담고 있으며, 세 계층이 각각 이 세 가지 관점에서 점수를 계산한다. 최종 관계 logits(re_prediction_scores)는

- 주어 기반 점수 m1_scores (batch × num_labels),
- (sub,obj) 기반 점수 m2_scores (batch × ent_len × num_labels),
- (sub,q) 기반 점수 m3_scores

를 더해 얻는다. qualifier 분류기도 동일한 구조로 q_re_classifier_m1/2/3를 사용해 qualifier logits(q_re_prediction_scores)를 계산하며, 각 (sub, obj, q) triple에 대해 qualifier 타입을 예측한다.

```
if self.nary_schema == "hyperrelation":
    outputs = (re_prediction_scores, ner_prediction_scores, q_re_prediction_scores, q_ner_prediction_scores)

    if labels is not None:
        loss_fct_re = CrossEntropyLoss(ignore_index=-1, weight=self.alpha.to(re_prediction_scores))
        loss_fct_ner = CrossEntropyLoss(ignore_index=-1, weight=self.ner_alpha.to(ner_prediction_scores))
        loss_fct_q_re = CrossEntropyLoss(ignore_index=-1, weight=self.q_alpha.to(q_re_prediction_scores))
        re_loss = loss_fct_re(re_prediction_scores.view(-1, self.num_labels), labels.view(-1))
        ner_loss = loss_fct_ner(ner_prediction_scores.view(-1, self.num_ner_labels), ner_labels.view(-1))
        q_re_loss = loss_fct_q_re(q_re_prediction_scores.view(-1, self.num_q_labels), q_labels.view(-1))
        loss = re_loss + ner_loss + q_re_loss
        outputs = (loss, re_loss, ner_loss, q_re_loss) + outputs
```

각 loss 함수는 weight 인자로 null-label과 그 외 라벨에 대한 가중치를 담은 텐서를 받는다 (self.alpha, self.ner_alpha, self.q_alpha). 이를 통해 null-label이 지나치게 많은 데이터에서 null 예측의 비중을 줄이고, 실제 relation/qualifier가 있는 샘플을 잘못 예측했을 때의 손실을 상대적으로

크게 부여한다. 학습 루프에서는 이 loss를 받아 loss.backward()를 호출하고 AdamW 옵티마이저로 파라미터를 업데이트한다.

```
def to_gran_format(result_file, label_file, output_file):
    resf = open(result_file, 'r')
    res_dict = json.load(resf)
    testf = open(label_file, "r")
    test_lines = testf.readlines()
    if os.path.exists(output_file):
        os.remove(output_file)
    rawf = open(output_file, "a")
    res_set = []
    for i in range(0, test_lines.__len__()):
        if str(i) not in res_dict.keys():
            test_dict = json.loads(test_lines[i])
            num_sens = len(test_dict["relations"])
            res_dict[str(i)] = []
            for k in range(num_sens):
                res_dict[str(i)].append([k, []])
    for i in range(0, test_lines.__len__()):
        hypers = []
        # res_set.append([])
        test_dict = json.loads(test_lines[i])
        sentence = test_dict["sentences"][0]
        for hyper_relation in res_dict[str(i)]:
            for hr in hyper_relation[1]:
                sub = ""
                obj = ""
                att = ""
                for index in range(hr[0][0], hr[0][1]):
                    sub = sub + sentence[index] + " "
                sub = sub + sentence[hr[0][1]]
                for index in range(hr[1][0], hr[1][1]):
                    obj = obj + sentence[index] + " "
                obj = obj + sentence[hr[1][1]]
                for index in range(hr[3][0], hr[3][1]):
                    att = att + sentence[index] + " "
                att = att + sentence[hr[3][1]]
                hyper = {"N": 3, "relation": hr[2], "subject": sub, "object": obj, hr[4]: [att]}
                hyper = json.dumps(hyper) + "\n"
                hypers.append(hyper)
        rawf.writelines(hypers)
        res_set.append(hypers)
    return res_set
```

학습이 끝난 뒤에는 6개 순열 각각의 확률 벡터를 다시 합쳐 하나의 3-ary에 대한 확률 벡터로 만든 뒤 3-ary hyper-relation JSON를 생성한다.

```

def compaction(res_set, result_comp_file):
    if os.path.exists(result_comp_file):
        os.remove(result_comp_file)
    resf_comp = open(result_comp_file, "a")
    res_table = []

    for res_line in res_set:
        res_comp_line = []
        # 用 map 将主三元组相同的超关系归到一类
        hy_map = {}
        for index in range(res_line.__len__()):
            res_dict = json.loads(res_line[index])
            rso = res_dict["relation"] + res_dict["subject"] + res_dict["object"]
            if rso in hy_map.keys():
                hy_map[rso].append(res_dict)
            else:
                hy_map[rso] = [res_dict]
        # 构建合并后的超关系
        for rso, ds in hy_map.items():
            t_d = {"N": 0}
            ext=0
            for d in ds:
                for k, v in d.items():
                    if k in t_d.keys() and k!="relation" and k!="subject" and k!="object" and k!="N":
                        t_d[k]+=v
                        ext+=1
                    else:
                        t_d[k]=v
            t_d["N"] = t_d.__len__() - 2 +ext
            res_comp_line.append(json.dumps(t_d))
        res_table.append(res_comp_line)
        formal_res_comp_line = []
        for hyper_relation in res_comp_line:
            formal_res_comp_line.append(hyper_relation + "\n")
        resf_comp.writelines(formal_res_comp_line)
    return res_table

```

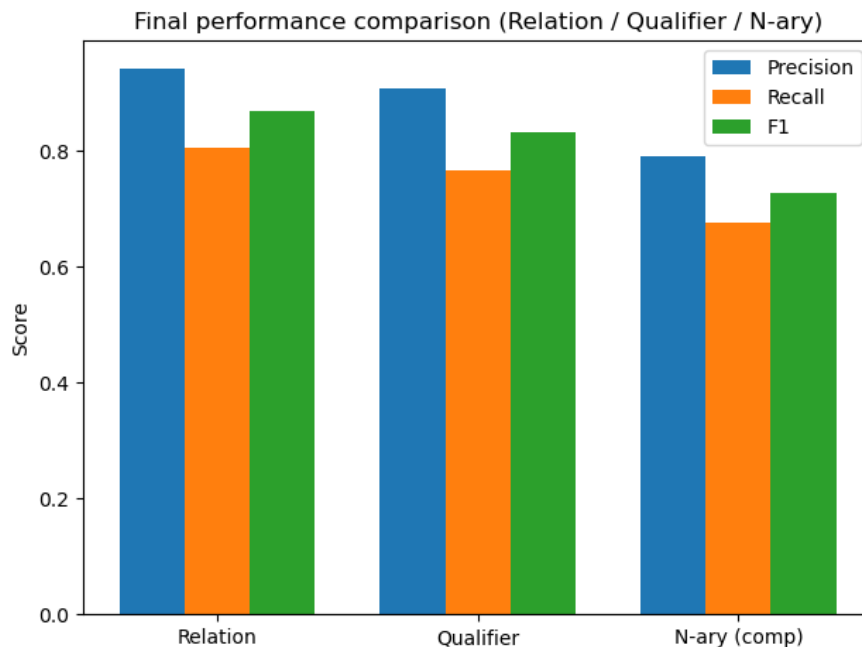
생성된 3-ary hyper-relation을 같은 (relation, subject, object)을 기준으로 묶어 최종 N-ary hyper-fact로 재조립하고, precision/recall/F1과 N(arity) 분포를 계산한다.

5. 실행 결과

result.json에 요약된 결과를 정리하면 다음과 같다.

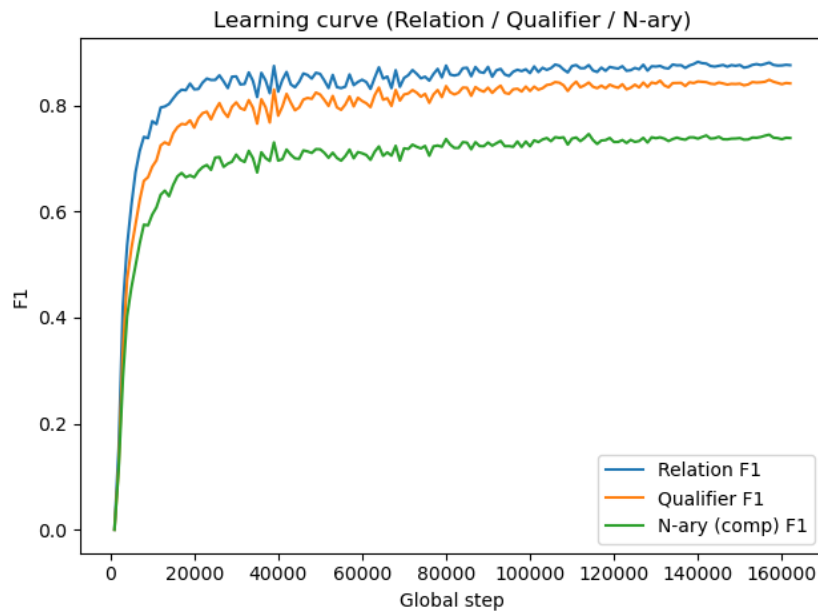
Metric	Value
Dev best F1	0.848268
Test relation F1	0.868825
Test relation Precision	0.94246
Test relation Recall	0.805863
Test qualifier F1	0.83152
Test qualifier Precision	0.908351
Test qualifier Recall	0.766673
Test N-ary(comp) F1	0.728197
Test N-ary(comp) Precision	0.790118
Test N-ary(comp) Recall	0.675277

정밀도(precision)에 비해 재현율(recall)이 상대적으로 낮으며, qualifier 예측은 전반적으로 relation보다 약간 더 어려운 태스크임을 알 수 있다. 모든 qualifier를 함께 고려하는 N-ary(comp) 기준에서는 예측 난이도가 가장 높아져 F1이 더 떨어지는 양상을 보인다. 이는 relation과 qualifier 중 하나라도 예측에 실패한 경우 N-ary 예측에 실패한 것으로 취급하기 때문이다.



Arity (N)	Num gold facts	Num predicted facts
3	4282	3709
4	550	425
5	41	29
7	4	0
10	1	0
6	0	6

대부분의 하이퍼팩트는 $N=3$ (기본적인 삼항 구조)에 해당하며, $N \geq 4$ 이상의 복잡한 fact는 상대적으로 매우 적다. 모델 역시 예측 시 주로 $N=3 \sim 4$ 범위의 fact를 생성하고 있음을 확인할 수 있다.



<experimental_data.json에 저장된 학습 곡선>

6. 결론 및 향후 과제

본 보고서에서는 Text2NKG의 공개 코드를 기반으로, HyperRED 데이터셋에 대한 코드 재현 및 실험을 수행하고 그 결과를 정리하였다. 복잡한 구조의 문장에 대해서는 N-ary 예측 성공률이 낮은 점이 아쉬우나, 논문에서 기존 re 모델과의 비교를 위해 BERT 기반 re 모델을 학습시켰으므로 LLM 기반 전처리/후처리 모듈을 도입해 문장에서 엔티티와 관계를 재추출하는 Gleaning을 수행하는 것으로 예측 성공률을 높일 수 있을 것으로 기대한다.